

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERIA

ESCUELA DE CIENCIAS Y SISTEMAS

ORGANIZACION DE LENGUAJES Y COMPILADORES 2

SECCIÓN A

MANUAL TECNICO

NOMBRE	CARNÉ
VICTOR HUGO VELASQUEZ	202100054
BRANDON EDUARDO PABLO GARCIA	202112092

Guatemala, junio del 2026

INTRODUCCION

TalentMosaic implementa un flujo de aprendizaje no supervisado para descubrir agrupaciones dentro de dos fuentes de datos independientes: el perfil de actividad de los freelancers y las reseñas textuales escritas por los clientes.

El sistema no combina ambos datasets antes del entrenamiento. Cada archivo sigue su propio proceso de limpieza, transformación y clustering. El campo `freelancer_id` se utiliza únicamente después de generar los clusters, con el objetivo de construir una tabla cruzada que relacione los segmentos de freelancers con los segmentos de reseñas.

Esta decisión se tomó porque ambos datasets representan dimensiones distintas del problema. El archivo `freelancers.csv` describe características cuantitativas y categóricas del profesional, mientras que `reseñas_clientes.csv` contiene información textual relacionada con la percepción del cliente. Mezclar ambas fuentes antes del entrenamiento podría provocar que una fuente domine a la otra, especialmente porque las reseñas se transforman en matrices de alta dimensionalidad mediante vectorización textual.

OBJETIVOS

- Brindar la información necesaria para poder representar la funcionalidad técnica de la estructura, diseño y definición del aplicativo.
- Describir las herramientas utilizadas para el diseño y desarrollo del prototipo

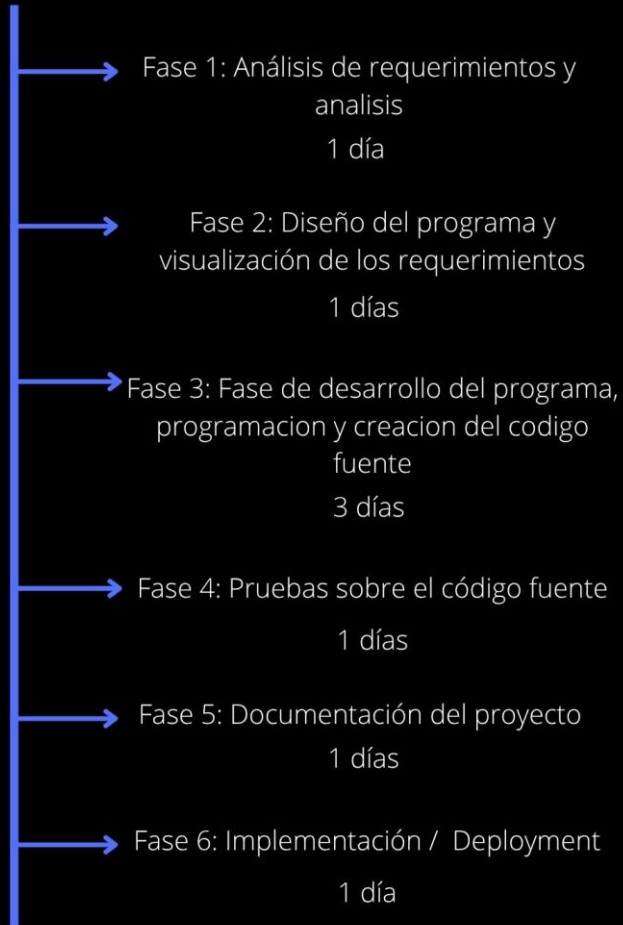
REQUERIMIENTOS DE FUNCION

Requerimientos	Descripción
Visual Studio Code	Se recomienda el uso VSC como ID (versión 1.109.5 superior) que fue la versión donde se programó el sistema de información
Angular	El frontend fue desarrollado utilizando el framework Angular (versión 15 o superior), el cual permite construir una interfaz web moderna, interactiva y basada en componentes.
Windows	El sistema fue desarrollado y probado en entorno Windows 10/11
Vite	Se recomienda el uso de Vite como empaquetador y servidor de desarrollo, por su rapidez en tiempo de compilación y recarga en caliente (HMR). Vite sustituye a Webpack en proyectos Angular modernos.

DESARROLLO

Agenda de proyecto

PROYECTO 1



CONTENIDO TECNICO

Backend

Análisis Exploratorio de Datos (EDA)

Freelancers.csv

El análisis exploratorio del dataset de freelancers tiene como objetivo identificar la estructura inicial de los datos, detectar valores faltantes, observar distribuciones, encontrar valores atípicos y analizar relaciones entre variables.

Las variables principales consideradas son:

- proyectos_completados
- ingresos_totales
- tarifa_hora_promedio
- anios_experiencia
- tiempo_respuesta_horas
- tasa_finalizacion
- calificacion_promedio
- categoria_principal
- clientes_recurrentes
- horas_trabajadas_mes

Para respaldar los hallazgos del EDA deben incluirse visualizaciones como histogramas, diagramas de caja, matriz de correlación y gráficos de barras.

Los histogramas permiten observar la distribución de las variables numéricas. Este análisis es necesario porque variables como ingresos_totales, proyectos_completados o clientes_recurrentes pueden presentar distribuciones sesgadas, donde pocos freelancers concentran valores muy altos. Este hallazgo justifica la necesidad de escalar los datos antes de aplicar algoritmos basados en distancia.

Los diagramas de caja permiten identificar valores atípicos. Este análisis es importante porque algoritmos como K-Means pueden verse afectados por registros extremos, ya que calculan centroides con base en distancias. Por esa razón, los valores atípicos no deben eliminarse automáticamente; primero debe analizarse si representan errores o freelancers realmente destacados dentro de la plataforma.

La matriz de correlación permite identificar relaciones entre variables numéricas. Por ejemplo, puede existir relación entre experiencia, ingresos, proyectos completados y clientes recurrentes. Esta visualización permite comprender qué variables podrían estar midiendo aspectos similares del desempeño del freelancer.

El gráfico de barras de categoria_principal permite observar la distribución de freelancers por área de servicio. Esto es importante porque una categoría sobrerrepresentada podría influir en la formación de clusters si no se transforma correctamente.

Para freelancers_dev.csv

freelancer_id	proyectos_completados	ingresos_totales	tarifa_hora_promedio	anios_experiencia	calificacion_promedio	categoria_principal
FL0001	42	73500.53	107.08	14.5	4.26	Desarrollo de Software
FL0002	37	4601.49	20.75	3.4	4.57	Diseño Gráfico
FL0003	9	2441.74	16.61	1.8	3.00	Diseño Gráfico
FL0005	19	30214.17	61.33	5.0	4.75	Marketing Digital

Reseñas_clientes.csv

El análisis exploratorio del dataset de reseñas se enfoca en la calidad del texto, longitud de comentarios, datos faltantes, duplicados y distribución por categoría de servicio.

Las variables principales consideradas son:

- reseña_id
- freelancer_id
- texto_reseña
- fecha_reseña
- categoria_servicio

- longitud_reseña

El análisis de longitud de reseñas permite identificar comentarios demasiado cortos o vacíos. Las reseñas muy cortas pueden no aportar suficiente información para el clustering textual, mientras que las reseñas muy largas pueden contener más términos y tener mayor peso en la representación vectorial.

El análisis de valores faltantes en texto_reseña es fundamental porque el proceso de vectorización requiere texto válido. Una reseña vacía no puede transformarse correctamente en una representación numérica útil.

El análisis de frecuencia de términos permite identificar palabras demasiado comunes. Este hallazgo justifica el uso de stopwords y de técnicas como TF-IDF, ya que no todas las palabras frecuentes aportan valor semántico para separar opiniones.

Para reseñas_clientes_dev.csv

reseña_id	freelancer_id	texto_reseña	fecha_reseña	categoria_servicio
RV00001	FL0001	La comunicación fue lo que más valoro...	2020-06-06	Desarrollo de Software
RV00002	FL0001	La calidad final fue buena aunque...	2021-05-14	Desarrollo de Software
RV00005	FL0001	El precio es razonable si se considera...	2023-02-12	Desarrollo de Software
RV00006	FL0001	El precio cobrado fue excesivo...	2021-03-06	Desarrollo de Software

Limpieza de Datos

La limpieza de datos se realizó de forma separada para los dos datasets del proyecto, debido a que cada archivo contiene información de naturaleza distinta. El archivo freelancers.csv contiene principalmente datos tabulares, es decir, variables

numéricas y categóricas relacionadas con el perfil del freelancer. En cambio, el archivo reseñas_clientes.csv contiene principalmente texto escrito por los clientes, por lo que requiere un proceso especial de limpieza textual antes de aplicar clustering.

Esta separación es importante porque no todos los datos pueden limpiarse de la misma manera. Las variables numéricas necesitan imputación y escalamiento, las variables categóricas necesitan codificación, y los textos necesitan normalización antes de convertirse en vectores numéricos.

Limpieza general aplicada a los datasets

Antes de aplicar transformaciones específicas, el sistema realiza una limpieza básica sobre los datos cargados desde los archivos CSV.

Primero, se normalizan los nombres de las columnas. Esto significa que los nombres se convierten a minúsculas, se eliminan espacios innecesarios al inicio o al final y los espacios internos se reemplazan por guiones bajos.

Por ejemplo:

Nombre original	Nombre normalizado
Freelancer ID	freelancer_id
Texto Reseña	texto_reseña
Tarifa Hora Promedio	tarifa_hora_promedio

Esta limpieza se realiza para evitar errores durante el procesamiento. Si un archivo viene con columnas escritas de forma diferente, por ejemplo, con mayúsculas o espacios, el sistema puede tener problemas para identificarlas. Al normalizar los nombres, se logra que el backend trabaje con una estructura más consistente.

Después de normalizar las columnas, se eliminan las filas completamente vacías. Esta decisión se justifica porque una fila sin ningún dato no aporta información útil para el clustering y puede generar errores en las etapas posteriores.

También se eliminan registros duplicados. Esto es importante porque, en aprendizaje no supervisado, si un mismo registro aparece repetido varias veces, puede afectar artificialmente la formación de los clusters. Un registro duplicado puede hacer que cierto patrón tenga más peso del que realmente debería tener.

En las columnas de tipo texto, el sistema elimina espacios innecesarios y convierte valores como cadenas vacías, "nan" o "None" en valores nulos reales. Esto permite que las estrategias de imputación trabajen correctamente más adelante.

Limpieza y preprocesamiento de freelancers.csv

El dataset freelancers.csv se procesa como un dataset tabular. Esto significa que el sistema identifica automáticamente qué columnas son numéricas y cuáles son categóricas.

Antes de entrenar el modelo, el sistema excluye columnas que no deben participar en el clustering, como los identificadores. Por ejemplo, columnas como freelancer_id, id, review_id, client_id o cliente_id no se usan como características del modelo.

Esta decisión es importante porque un identificador no representa comportamiento, desempeño ni características reales del freelancer. Si se incluyera freelancer_id en el entrenamiento, el modelo podría formar clusters con base en un número arbitrario, lo cual no tendría valor analítico.

También se excluyen columnas de texto largo, como reseñas, comentarios o descripciones, porque ese tipo de información requiere otro proceso diferente basado en vectorización de texto.

Tratamiento de valores faltantes en variables numéricas

Para las variables numéricas del dataset de freelancers, el sistema utiliza imputación con la mediana.

Esto se aplica mediante:

SimpleImputer(strategy="median")

La mediana fue seleccionada porque es más resistente a valores extremos que la media. En este dataset pueden existir variables con valores muy dispersos, por ejemplo:

- ingresos_totales
- proyectos_completados
- tarifa_hora_promedio
- clientes_recurrentes
- horas_trabajadas_mes
-

En este tipo de variables, algunos freelancers pueden tener ingresos o proyectos mucho más altos que el resto. Si se usara la media, esos valores extremos podrían

alterar demasiado el valor imputado. En cambio, la mediana representa mejor el comportamiento central de los datos cuando hay posibles outliers.

Esta estrategia se eligió sobre eliminar filas porque eliminar registros con valores faltantes puede reducir la cantidad de datos disponibles para el entrenamiento. En clustering, conservar la mayor cantidad posible de observaciones útiles ayuda a formar segmentos más representativos.

Escalamiento de variables numéricas

Después de imputar valores faltantes, las variables numéricas se escalan usando `StandardScaler`.

Esto se aplica mediante:

`StandardScaler()`

El escalamiento es necesario porque las variables del dataset tienen rangos muy diferentes. Por ejemplo, `ingresos_totales` puede tener valores de cientos o miles de dólares, mientras que `calificacion_promedio` está en una escala de 0 a 5.

Si no se escalaran los datos, las variables con valores numéricos más grandes dominarían el cálculo de distancias. Esto afectaría especialmente a algoritmos como K-Means, DBSCAN o clustering aglomerativo, ya que estos dependen de la distancia entre registros.

Por esa razón, el escalamiento permite que todas las variables numéricas participen de forma más equilibrada en el entrenamiento del modelo.

Tratamiento de variables categóricas

Las variables categóricas, como `categoria_principal`, se procesan de forma distinta a las numéricas.

Primero, los valores faltantes se rellenan con la palabra "desconocido".

Esto se aplica mediante:

`SimpleImputer(strategy="constant", fill_value="desconocido")`

Esta estrategia permite conservar registros, aunque no tengan categoría asignada. En vez de eliminar esos freelancers, el sistema los mantiene y los agrupa bajo una categoría especial. Esto es útil porque la ausencia de una categoría también puede ser información relevante.

Luego, las variables categóricas se transforman usando One-Hot Encoding.

Esto se aplica mediante:

OneHotEncoder(handle_unknown="ignore")

One-Hot Encoding convierte cada categoría en una columna binaria. Por ejemplo, si existen categorías como Desarrollo de Software, Diseño Gráfico y Marketing Digital, cada una se representa como una columna separada con valores 0 o 1.

Esta técnica fue seleccionada porque las categorías no tienen un orden natural. No sería correcto convertirlas en números como 1, 2 o 3, porque eso podría hacer que el modelo interprete una falsa jerarquía entre categorías.

Además, se utiliza `handle_unknown="ignore"` para evitar errores cuando se clasifica un nuevo freelancer con una categoría que no apareció durante el entrenamiento. En ese caso, el sistema simplemente ignora esa categoría desconocida sin romper el proceso.

Resultado final del preprocesamiento tabular

Después de limpiar e imputar los datos, el sistema construye una matriz numérica lista para clustering.

El resultado incluye:

- Datos numéricos imputados y escalados.
- Variables categóricas imputadas y codificadas.
- Identificadores conservados aparte, si existen.
- Nombres finales de las características.
- Dataset limpio usado como base para interpretación.

Esto permite que el modelo trabaje únicamente con variables útiles y comparables entre sí.

Limpieza y preprocesamiento de reseñas_clientes.csv

El dataset de reseñas requiere una limpieza diferente porque su variable principal es textual. El objetivo de esta limpieza es reducir ruido y convertir los comentarios en una forma más uniforme antes de vectorizarlos.

Primero, el sistema normaliza los nombres de columnas y elimina filas vacías o duplicadas. Luego identifica automáticamente la columna que contiene el texto de la reseña.

El sistema busca nombres comunes como:

- `review_text`

- review
- reseña
- resena
- comentario
- comentarios
- texto
- description
- descripción

Esto permite que el sistema sea flexible, aunque el archivo CSV venga con nombres de columnas ligeramente diferentes.

Normalización del texto

Cada reseña se limpia mediante una función de normalización textual.

El proceso aplicado es el siguiente:

1. Convierte el valor a texto.
2. Convierte todo a minúsculas.
3. Elimina tildes y marcas Unicode.
4. Elimina URLs.
5. Elimina caracteres especiales.
6. Normaliza espacios múltiples.

Por ejemplo, un texto como:

"Excelente comunicación!!! Entregó rápido y muy bien."

puede quedar como:

"excelente comunicacion entrego rapido y muy bien"

Esta limpieza es necesaria porque el modelo no entiende el significado del texto directamente. Primero necesita convertir las palabras en una representación numérica. Si no se normaliza el texto, palabras equivalentes podrían quedar separadas como términos distintos.

Por ejemplo:

Texto original	Después de limpiar
Comunicación	comunicacion

COMUNICACION	comunicacion
comunicación!!!	comunicacion

Esto ayuda a reducir la cantidad de términos repetidos o innecesarios en el vocabulario.

Eliminación de URLs y caracteres especiales

El sistema elimina enlaces como páginas web o URLs porque generalmente no aportan información semántica útil para clasificar la reseña.

También elimina signos de puntuación y caracteres especiales. Esto reduce el ruido textual y evita que el vectorizador trate símbolos como si fueran parte importante del contenido.

Esta estrategia fue preferida porque el objetivo del modelo es encontrar temas recurrentes en las opiniones, no analizar formato, enlaces o signos de puntuación.

Manejo de reseñas vacías

Después de limpiar el texto, el sistema valida si la columna completa quedó vacía. Si todos los textos están vacíos después de la limpieza, se lanza un error y no se permite continuar con la vectorización.

Esta validación es importante porque no se puede entrenar un modelo textual si no existe contenido real para transformar en vectores.

Sin embargo, con el código actual, si solo algunas reseñas individuales quedan vacías, no se eliminan automáticamente en esta función. Es recomendable revisar este punto, porque una reseña vacía puede generar un vector sin información útil y afectar el clustering.

Una mejora posible sería eliminar registros donde texto_reseña quede vacío después de aplicar la limpieza textual.

Stopwords en español

El sistema utiliza una lista de stopwords en español. Estas son palabras comunes que normalmente no aportan mucho significado para diferenciar reseñas.

Algunos ejemplos son:

- el
- la
- los
- de
- y
- en
- por
- para
- muy
- que

Eliminar estas palabras es importante porque aparecen con mucha frecuencia, pero no necesariamente ayudan a descubrir temas. Si se conservaran, podrían provocar que las reseñas se agrupen por palabras gramaticales en lugar de agruparse por contenido real.

Vectorización después de la limpieza

Luego de limpiar el texto, el sistema lo transforma usando un vectorizador. La opción principal es TF-IDF, aunque también existe la opción de Bag of Words.

La limpieza textual es necesaria antes de esta etapa porque el vectorizador construye un vocabulario a partir de las palabras encontradas. Si el texto no se limpia, el vocabulario puede llenarse de términos repetidos, errores de formato, signos o palabras poco útiles.

Por ejemplo, sin limpieza el sistema podría interpretar como palabras distintas:

- Excelente
- excelente
- excelente!
- EXCELENTE

Después de limpiar, todas quedan representadas como:

excelente

Esto mejora la consistencia de la matriz textual y ayuda a que los clusters sean más interpretables.

Justificación general de las decisiones de limpieza

Las estrategias de limpieza utilizadas buscan conservar la mayor cantidad posible de información útil sin introducir ruido innecesario.

En el dataset de freelancers, se evita eliminar registros completos cuando tienen algunos valores faltantes. En su lugar, se imputan valores numéricos con la mediana y valores categóricos con "desconocido". Esta decisión permite mantener más datos para el entrenamiento y reduce el riesgo de perder perfiles importantes.

En el dataset de reseñas, se prioriza la normalización del texto para que comentarios escritos de formas distintas puedan ser comparados correctamente. La eliminación de tildes, URLs, caracteres especiales y espacios múltiples permite obtener una representación más limpia antes de aplicar TF-IDF o Bag of Words.

Además, se eliminan duplicados porque los registros repetidos pueden alterar la distribución real de los datos y afectar los clusters. También se excluyen identificadores como `freelancer_id` del entrenamiento, porque estos campos no describen comportamiento ni contenido, sino que solo sirven para relacionar datos después del clustering.

En conclusión, la limpieza aplicada permite transformar datos crudos en una base más consistente, comparable y adecuada para aprendizaje no supervisado.

Vectorización de Texto

Para transformar las reseñas en datos numéricos, el sistema permite utilizar dos técnicas:

- TF-IDF.
- Bag of Words.

La técnica por defecto es TF-IDF, implementada mediante `TfidfVectorizer`.

TF-IDF fue seleccionada como opción principal porque no solo considera cuántas veces aparece una palabra, sino también qué tan distintiva es dentro del conjunto completo de reseñas. Esto es útil porque algunas palabras pueden aparecer muchas veces, pero no necesariamente ayudan a diferenciar temas entre clientes.

Bag of Words se mantiene como alternativa porque permite una representación más simple basada en conteos directos de términos. Sin embargo, para clustering de reseñas, TF-IDF suele ser más adecuada porque reduce el peso de términos excesivamente frecuentes.

La configuración utilizada para TF-IDF es:

- `max_features=1000`

- `gram_range=(1, 2)`
- `min_df=1`
- `stop_words=STOPWORDS_ES`

El parámetro `max_features=1000` limita el vocabulario a los 1000 términos más relevantes. Esta decisión evita crear una matriz demasiado grande y reduce el ruido generado por palabras poco informativas.

El parámetro `gram_range=(1, 2)` permite capturar palabras individuales y combinaciones de dos palabras. Esto es importante porque en reseñas de clientes muchas ideas se expresan mediante frases cortas, por ejemplo “buena comunicación”, “entrega tarde” o “excelente trabajo”. Usar únicamente palabras individuales podría perder parte de ese contexto.

El parámetro `min_df=1` permite conservar términos que aparecen al menos una vez. Esta configuración es flexible para datasets pequeños o medianos, ya que evita eliminar términos que podrían ser importantes en comentarios específicos. En datasets grandes podría evaluarse aumentar este valor para reducir ruido.

La lista `STOPWORDS_ES` elimina palabras frecuentes del español que no aportan significado suficiente, como artículos, conectores y preposiciones. Esta decisión mejora la representación textual porque evita que las reseñas se agrupen por palabras gramaticales en lugar de temas reales.

Selección del Modelo

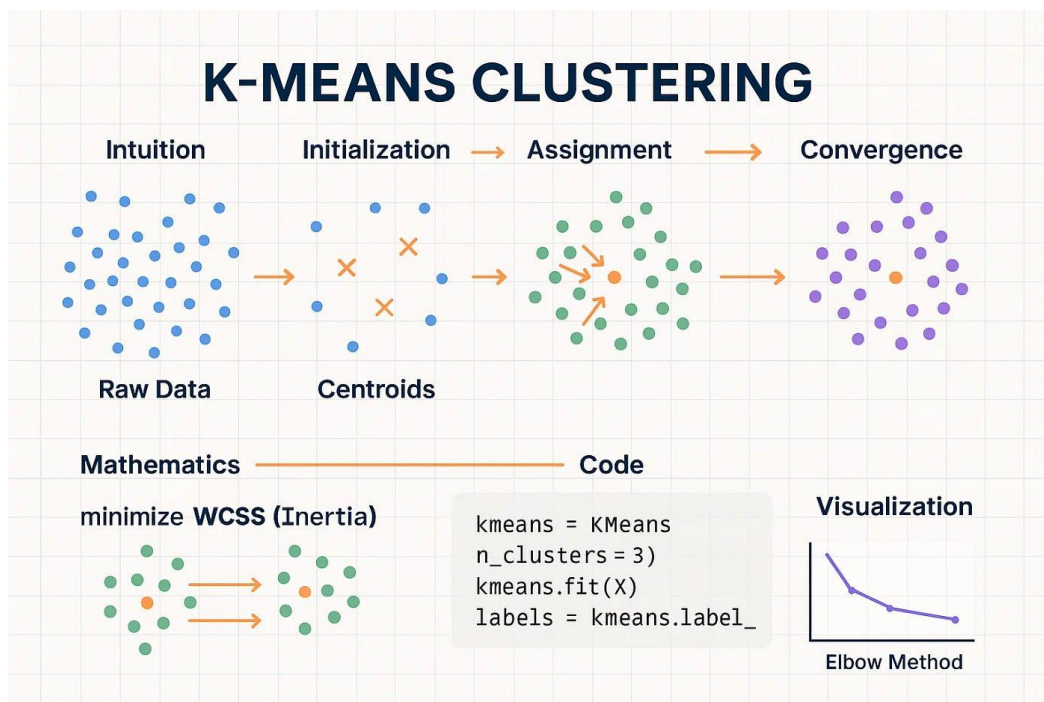
El sistema contempla cuatro algoritmos de clustering:

- K-Means.
- DBSCAN.
- Clustering Jerárquico Aglomerativo.
- Gaussian Mixture Model.

Esta variedad permite adaptar el entrenamiento al tipo de estructura encontrada en los datos.

K-Means

K-Means se utiliza como algoritmo principal por defecto. Este algoritmo agrupa observaciones alrededor de centroides y es adecuado cuando se espera formar segmentos relativamente compactos y comparables entre sí.



En esta prueba se configuró el modelo con los siguientes parámetros:

Parámetros de entrenamiento

Configura el dataset, algoritmo y parámetros del modelo de clustering.

Tipo de dataset

Freelancers



Nombre del archivo

freelancers_dev.csv

Algoritmo

K-Means



Random state

42

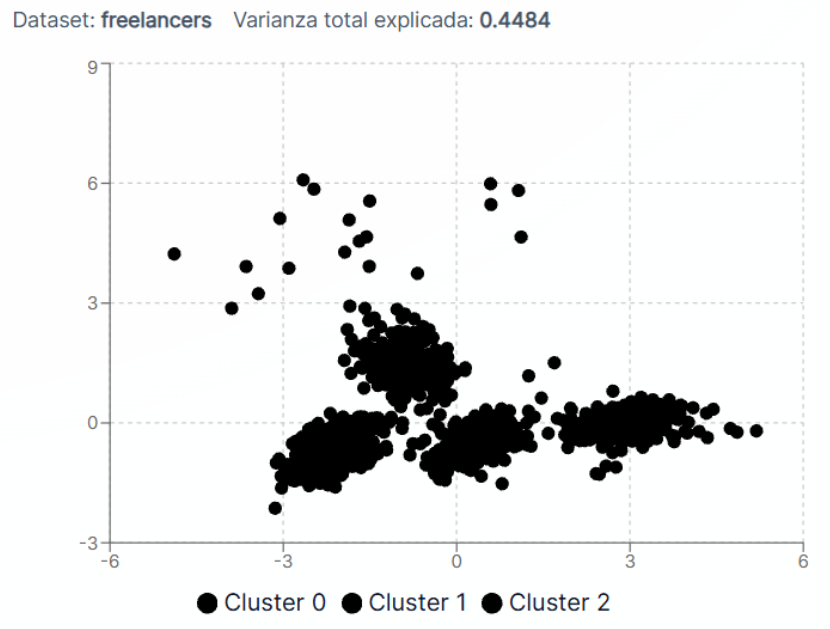
Número de clusters

3

El valor de `n_clusters=3` se utilizó para generar tres segmentos iniciales de freelancers. Esta configuración permite obtener una segmentación sencilla de interpretar, evitando crear demasiados grupos pequeños que dificulten el análisis. Además, el uso de `random_state=42` garantiza que el entrenamiento sea reproducible, es decir, que al ejecutar nuevamente el modelo con los mismos datos se obtengan resultados consistentes.

El modelo generó 3 clusters y no produjo puntos de ruido. Esto es esperado en K-Means, ya que este algoritmo siempre asigna cada registro a un cluster. A diferencia de

DBSCAN, K-Means no marca observaciones como ruido, sino que clasifica todos los datos según el centroide más cercano.



Las métricas obtenidas fueron las siguientes:

Silhouette Score

0.1964

Más alto es mejor.

Davies-Bouldin

1.7706

Más bajo es mejor.

Calinski-Harabasz

390.9571

Más alto es mejor.

Varianza explicada PCA/SVD

0.3407 + 0.1076

El modelo K-Means obtuvo un Silhouette Score de 0.1964, lo que indica una separación débil entre los clusters. Esto significa que algunos freelancers comparten características similares entre segmentos, por lo que los grupos no están completamente separados.

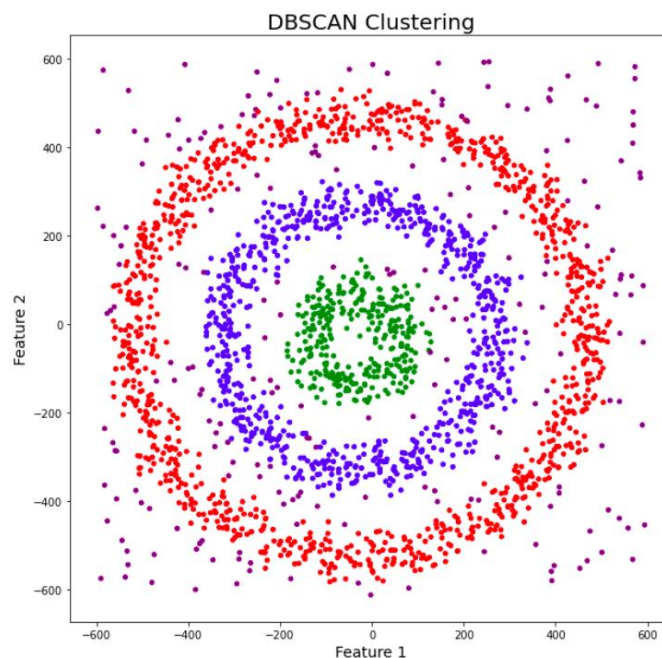
El índice Davies-Bouldin fue de 1.7706, lo cual confirma cierta cercanía entre los clusters. Por su parte, el Calinski-Harabasz fue de 390.9571, valor que se toma como referencia para comparar este modelo contra otros algoritmos o distintos valores de k.

La varianza explicada por PCA/SVD fue de 0.3407 y 0.1076, sumando aproximadamente 44.83%. Esto indica que la visualización en dos dimensiones es útil como apoyo gráfico, aunque no representa toda la información del dataset.

Aunque las métricas no muestran una separación fuerte, K-Means sigue siendo una buena opción base porque es eficiente, fácil de interpretar, permite definir directamente el número de segmentos y posee método predict para clasificar nuevos freelancers. Por ello, se utiliza como modelo inicial de comparación frente a otros algoritmos.

DBSCAN

DBSCAN se considera como alternativa cuando los datos pueden contener ruido o clusters con formas no necesariamente esféricas. A diferencia de K-Means, DBSCAN no requiere definir el número de clusters previamente.



En esta prueba se configuró el modelo con los siguientes parámetros:

Parámetros de entrenamiento

Configura el dataset, algoritmo y parámetros del modelo de clustering.

Tipo de dataset

Freelancers



Nombre del archivo

freelancers_dev.csv

Algoritmo

DBSCAN



Random state

42

EPS

0.5

Min samples

5

La primera prueba con DBSCAN usando $\text{eps}=0.5$ y $\text{min_samples}=5$ no generó clusters válidos, ya que todos los registros fueron clasificados como ruido. Esto indica que el radio de vecindad fue demasiado restrictivo para la estructura del dataset.

Registros

1500

Features

1820

Clusters

0

Ruido

1500

Silhouette Score

No disponible

Más alto es mejor.

Davies-Bouldin

No disponible

Más bajo es mejor.

Calinski-Harabasz

No disponible

Más alto es mejor.

Varianza explicada PCA/SVD

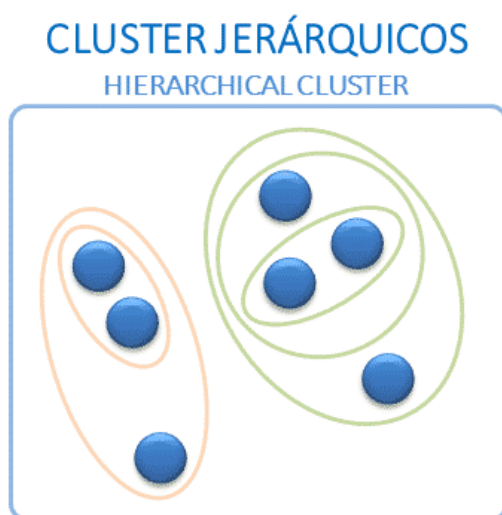
0.3407 + 0.1076

Este comportamiento indica que, con $\text{eps}=0.5$ y $\text{min_samples}=5$, el modelo fue demasiado restrictivo para este dataset. Además, al trabajar con 1820 features, la distancia entre registros puede dificultar la detección de zonas densas.

Por lo tanto, DBSCAN no fue seleccionado como modelo final en esta configuración, ya que no produjo segmentos interpretables ni permitió comparar adecuadamente la calidad del agrupamiento.

Clustering Jerárquico Aglomerativo

El clustering aglomerativo se considera como alternativa para analizar estructuras jerárquicas entre registros. Este algoritmo inicia considerando cada observación como un cluster individual y luego va fusionando grupos según un criterio de enlace.



En esta prueba se configuró el modelo con los siguientes parámetros:

Parámetros de entrenamiento

Configura el dataset, algoritmo y parámetros del modelo de clustering.

Tipo de dataset

Freelancers



Nombre del archivo

freelancers_dev.csv

Algoritmo

Clustering jerárquico



Random state

42

Número de clusters

3

Linkage

ward



El modelo logró formar 3 clusters y no generó ruido. Sin embargo, el Silhouette Score de 0.1627 indica una separación débil entre los grupos. Además, el Davies-Bouldin de

1.9110 es mayor que el obtenido con K-Means, lo que sugiere mayor cercanía o solapamiento entre clusters.

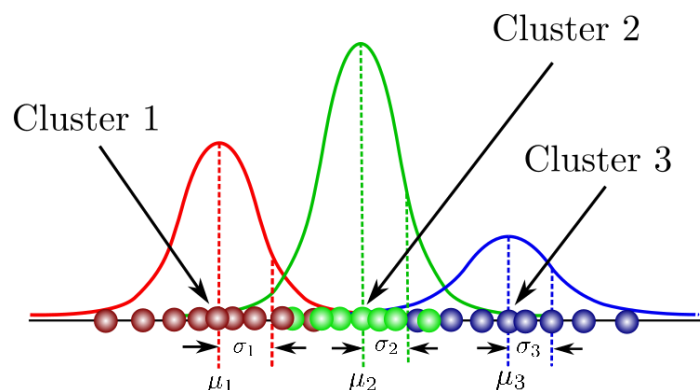
El Calinski-Harabasz fue de 382.5461, ligeramente menor que el de K-Means, por lo que esta configuración no mostró una mejora en la calidad del agrupamiento



Aunque el clustering jerárquico permite analizar la estructura de los datos, en esta prueba no superó a K-Means. Además, no posee método predictivo nativo, por lo que requiere un modelo auxiliar para clasificar nuevos registros. Por esta razón, no se seleccionó como modelo final.

Gaussian Mixture Model

Gaussian Mixture Model se incluye como alternativa probabilística. A diferencia de K-Means, GMM permite modelar clusters con distintas formas y asignar registros según componentes gaussianas.



En esta prueba se configuró el modelo con los siguientes parámetros:

Parámetros de entrenamiento

Configura el dataset, algoritmo y parámetros del modelo de clustering.

Tipo de dataset

Freelancers



Nombre del archivo

freelancers_dev.csv

Algoritmo

GMM



Random state

42

Número de clusters

3

El modelo logró formar 3 clusters y no generó ruido. Sin embargo, el Silhouette Score de 0.1698 indica una separación débil entre los grupos, menor que la obtenida con K-Means. El Davies-Bouldin de 1.8981 también fue mayor que el de K-Means, lo que indica mayor similitud entre clusters.

El Calinski-Harabasz fue de 388.8826, ligeramente menor que K-Means, por lo que tampoco presentó una mejora clara en la compactación y separación de los segmentos.

Registros

1500

Features

1820

Clusters

3

Ruido

0

Silhouette Score

0.1698

Más alto es mejor.

Davies-Bouldin

1.8981

Más bajo es mejor.

Calinski-Harabasz

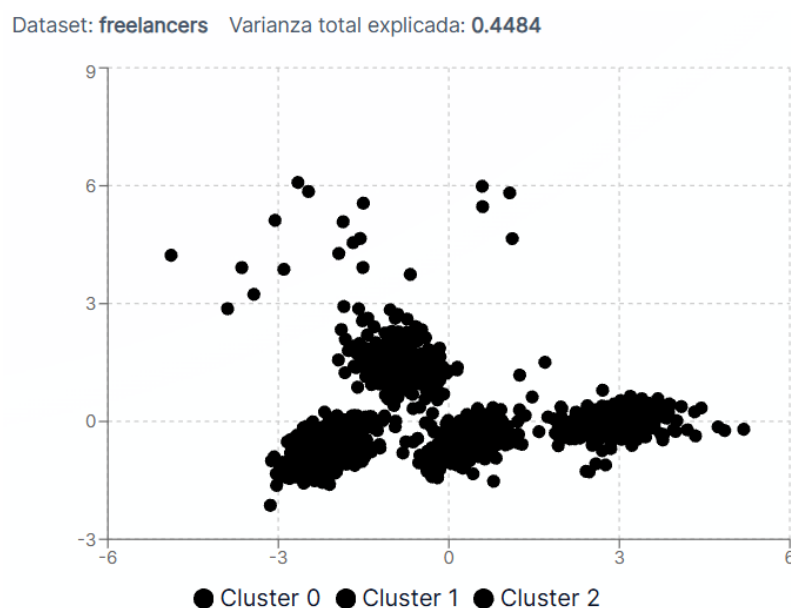
388.8826

Más alto es mejor.

Varianza explicada PCA/SVD

0.3407 + 0.1076

Aunque GMM permite un enfoque más flexible que K-Means, en esta prueba no superó sus métricas. Por ello, no se seleccionó como modelo final para este dataset.



Justificación del modelo seleccionado

El modelo final debe seleccionarse considerando métricas internas, número de clusters encontrados, ruido, interpretabilidad y capacidad de clasificar nuevos registros.

Para datos tabulares de freelancers, K-Means suele ser una opción adecuada cuando las variables han sido escaladas correctamente y se busca interpretar perfiles promedio por segmento.

Para reseñas vectorizadas con TF-IDF, K-Means también resulta apropiado porque trabaja bien con representaciones numéricas de alta dimensionalidad y permite interpretar clusters a partir de términos representativos.

No obstante, el sistema conserva DBSCAN, clustering aglomerativo y GMM como alternativas para comparar resultados y justificar técnicamente la selección final.

Entrenamiento y Ajuste de Parámetros

El entrenamiento se controla mediante el esquema TrainingRequest, que permite configurar el tipo de dataset, archivo, algoritmo y parámetros principales.

Los parámetros soportados son:

- algorithm

- n_clusters
- eps
- min_samples
- linkage
- text_column
- vectorizer
- random_state

Para K-Means, Agglomerative y GMM se exploran distintos valores de n_clusters, entre 2 y 20. Esta exploración es necesaria porque el número de clusters afecta directamente la interpretación de los segmentos.

Un número bajo de clusters puede generar segmentos demasiado generales. Por el contrario, un número alto puede crear grupos pequeños, difíciles de interpretar o poco útiles para la toma de decisiones.

Para DBSCAN, el sistema incluye el cálculo de la curva de distancia al k-ésimo vecino. Esta curva ayuda a seleccionar eps, ya que permite identificar un punto donde las distancias empiezan a aumentar con mayor rapidez.

La configuración final debe elegirse con base en:

- Mejor equilibrio entre métricas internas.
- Cantidad razonable de clusters.
- Bajo número de puntos de ruido.
- Facilidad de interpretación de los segmentos.
- Capacidad de clasificar registros nuevos.

Evaluación del Modelo

La evaluación se realiza mediante métricas internas de clustering, implementadas en `compute_clustering_metrics`.

Las métricas utilizadas son:

- Silhouette Score.
- Davies-Bouldin Score.
- Calinski-Harabasz Score.

Estas métricas son adecuadas porque el aprendizaje no supervisado no cuenta con etiquetas reales para comparar predicciones.

Comparación de modelos entrenados

Usa esta tabla para comparar configuraciones y decidir cuál modelo tiene mejor comportamiento.

Dataset	Algoritmo	Archivo	Clusters	Ruido	Silhouette	Davies-Bouldin	Calinski-Harabasz	Model ID
freelancers	gmm	freelancers_dev.csv	3	0	0.1698	1.8981	388.8826	024be3ef-8748-451a-baa9-85db597ee5d6
freelancers	agglomerative	freelancers_dev.csv	3	0	0.1627	1.9110	382.5461	cbb37526-564e-4e00-aa48-12cbda0f7ab7
freelancers	dbscan	freelancers_dev.csv	0	1500	N/A	N/A	N/A	59f28ad3-cbfc-4fe2-9f95-d38ced952404
freelancers	kmeans	freelancers_dev.csv	3	0	0.1964	1.7706	390.9571	bb67a2c5-2947-4db1-9301-cda0567bb0cf

Silhouette Score

El Silhouette Score mide qué tan similar es una observación a su propio cluster en comparación con otros clusters. Valores más altos indican mejor separación.

El sistema interpreta esta métrica de la siguiente manera:

- Mayor o igual a 0.70: muy buena separación.
- Mayor o igual a 0.50: buena separación.
- Mayor o igual a 0.25: separación moderada.
- Menor a 0.25: separación débil.

Esta métrica se utiliza porque permite evaluar tanto compactación interna como separación externa.

Silhouette Score

Débil

0.1698

Mide qué tan bien separado está cada punto de otros clusters.
Más alto es mejor.

Los clusters están poco separados o se mezclan entre sí.

Davies-Bouldin Score

El Davies-Bouldin Score mide la similitud entre clusters. En esta métrica, valores más bajos son mejores, ya que indican que los clusters son más diferentes entre sí.

Esta métrica complementa al Silhouette Score porque permite identificar si los clusters están demasiado cerca o si presentan alta dispersión interna.

Davies-Bouldin Score

Aceptable

1.8981

Mide similitud entre clusters. Más bajo es mejor.

La separación es razonable, aunque puede mejorar.

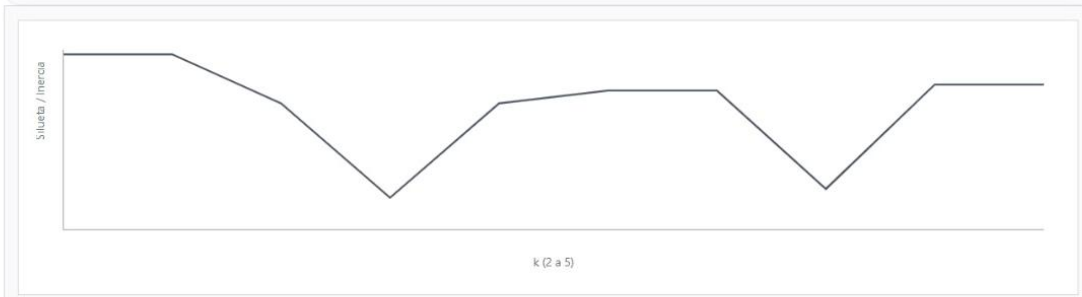
Calinski-Harabasz Score

El Calinski-Harabasz Score evalúa la relación entre la dispersión entre clusters y la dispersión interna de cada cluster. Valores más altos indican agrupaciones más definidas.

Esta métrica se utiliza como apoyo para comparar configuraciones con distintos valores de k.

Que representa esta grafica

Los picos aparecen al comparar varios modelos entrenados con diferentes valores de k. No representan clusters individuales.



Tratamiento del ruido en evaluación

En el caso de DBSCAN, la etiqueta -1 representa ruido. El sistema excluye estos puntos al calcular métricas internas. Esta decisión es correcta porque los puntos de ruido no pertenecen a un cluster válido y podrían distorsionar la evaluación.

El sistema también valida que existan al menos dos clusters válidos antes de calcular métricas. Si solo existe un cluster o si cada punto queda como su propio cluster, las métricas se devuelven como nulas porque no serían interpretables.

Reducción Dimensional para Visualización

Para visualizar los clusters en dos dimensiones, el sistema utiliza la función `reduce_to_2d`.

Cuando la matriz de entrada es densa, se utiliza PCA. PCA permite proyectar los datos en dos componentes principales que conservan la mayor cantidad posible de varianza.

Cuando la matriz es dispersa, como ocurre con TF-IDF o Bag of Words, se utiliza TruncatedSVD. Esta decisión es necesaria porque PCA no trabaja eficientemente con matrices sparse, mientras que TruncatedSVD sí es adecuado para datos textuales vectorizados.

El sistema devuelve:

- Coordenadas en dos dimensiones.
- Varianza explicada por cada componente.
- Varianza total explicada.
- Método utilizado.

Esta visualización no se utiliza para entrenar el modelo, sino para facilitar la interpretación gráfica de los clusters en el frontend o en reportes.

Interpretación de Segmentos

Segmentos de freelancers

Los segmentos de freelancers se interpretan usando las variables numéricas originales y las etiquetas de cluster generadas por el modelo.

El sistema construye nombres descriptivos mediante la función `build_numeric_segment_names`. Esta función compara el promedio global de cada variable contra el promedio de cada cluster. Luego calcula una diferencia estandarizada usando z-score y selecciona las variables más distintivas.

Por ejemplo, un cluster puede recibir un nombre como:

- Alto ingresos totales / Alto clientes recurrentes
- Bajo tiempo respuesta horas / Alto calificación promedio
- Alto tarifa hora promedio / Bajo proyectos completados

Tabla cruzada de segmentos

Registros cruzados: 1500

Tabla cruzada generada correctamente.

Conteos absolutos

Segmento freelancer	Reseña cluster 0	Reseña cluster 1	Reseña cluster 2
0	296	0	0
1	0	645	0
2	0	0	559

Porcentajes por fila

Segmento freelancer	Reseña cluster 0	Reseña cluster 1	Reseña cluster 2
0	100%	0%	0%
1	0%	100%	0%
2	0%	0%	100%

Esta estrategia es válida porque el número de cluster por sí solo no tiene significado. Lo importante es interpretar qué variables diferencian a cada grupo.

Segmentos de reseñas

Los segmentos de reseñas se interpretan con base en los términos más representativos de cada cluster.

La función `get_top_terms_by_cluster` calcula el peso promedio de cada término dentro de un cluster y selecciona los términos con mayor peso. Esto permite identificar los temas principales de cada segmento textual.

Luego, la función `build_text_segment_names` genera nombres cortos para los clusters usando los términos más relevantes.

Por ejemplo, un segmento puede nombrarse como:

- Calidad / Entrega
- Comunicacion / Tiempo
- Excelente / Servicio

Tabla cruzada de segmentos

Registros cruzados: 1500

Tabla cruzada generada correctamente.

Conteos absolutos

Segmento freelancer	Reseña cluster 0	Reseña cluster 1	Reseña cluster 2
0	296	0	0
1	0	645	0
2	0	0	559

Porcentajes por fila

Segmento freelancer	Reseña cluster 0	Reseña cluster 1	Reseña cluster 2
0	100%	0%	0%
1	0%	100%	0%
2	0%	0%	100%

Esta estrategia permite transformar clusters numéricos en categorías interpretables para el usuario final.

Tabla Cruzada entre Segmentos

Después de entrenar ambos modelos de forma independiente, el sistema utiliza `freelancer_id` para relacionar los segmentos de freelancers con los segmentos de reseñas.

La función `build_cross_segment_table` realiza una unión interna entre ambos resultados usando `freelancer_id`. Luego genera:

- Conteos absolutos.
- Porcentajes por fila.

Los conteos absolutos permiten saber cuántas reseñas de cada tipo corresponden a cada segmento de freelancer. Los porcentajes por fila permiten interpretar la distribución relativa de reseñas dentro de cada grupo de freelancers.

Esta tabla es importante porque permite responder preguntas como:

- ¿Qué tipo de freelancers reciben más reseñas positivas?
- ¿Qué segmentos reciben comentarios relacionados con tiempos de entrega?
- ¿Qué perfiles están asociados a problemas de comunicación?
- ¿Qué segmentos muestran mayor satisfacción del cliente?

Es importante aclarar que esta tabla no se usa para entrenar los modelos. Se utiliza únicamente para análisis e interpretación posterior.

Clasificación de Nuevos Registros

11.1 Nuevos freelancers

Para clasificar un nuevo freelancer, el sistema aplica el mismo preprocesador entrenado durante la etapa de entrenamiento.

La función `transform_tabular_record` recibe un nuevo registro, normaliza sus columnas y lo transforma usando el `ColumnTransformer` ya ajustado. Esto garantiza que el nuevo registro tenga las mismas columnas, escalas y codificación que los datos originales.

Después de transformar el registro, la función `predict_cluster` asigna el cluster correspondiente.

Si el modelo seleccionado tiene método `predict`, como K-Means o GMM, se utiliza directamente la predicción nativa del algoritmo.

Esta estrategia es válida porque el nuevo registro se ubica en el mismo espacio numérico usado durante el entrenamiento.

Nuevas reseñas

Para clasificar una nueva reseña, el sistema aplica el mismo proceso textual utilizado en entrenamiento.

La función `transform_text_record` normaliza el nombre de la columna, limpia el texto con `normalize_text` y transforma la reseña usando el vectorizador ya entrenado.

Es fundamental reutilizar el mismo vectorizador porque el vocabulario debe coincidir exactamente con el utilizado durante el entrenamiento. Entrenar un nuevo vectorizador para cada reseña generaría espacios de características diferentes y las predicciones serían inconsistentes.

Después de vectorizar la reseña, el modelo asigna el cluster correspondiente.

Clasificación con algoritmos sin predict nativo

Algunos algoritmos, como DBSCAN y Clustering Aglomerativo, no poseen método `predict` nativo.

Para resolver este problema, el sistema crea un modelo auxiliar basado en KNN mediante la función `build_assignment_model`.

Este modelo KNN se entrena usando los datos ya transformados y las etiquetas generadas por el clustering. En el caso de DBSCAN, se excluyen los puntos con etiqueta -1, ya que representan ruido.

Cuando llega un nuevo registro, el KNN auxiliar lo asigna al cluster más cercano según sus vecinos más próximos.

Esta estrategia es una aproximación válida porque permite reutilizar la estructura de clusters encontrada durante el entrenamiento, aunque el algoritmo original no permita clasificar nuevas observaciones directamente.

Serialización y Recuperación de Artefactos

Para que el sistema pueda clasificar nuevos registros después del entrenamiento, es necesario guardar no solo el modelo de clustering, sino también todos los artefactos de preprocesamiento.

Los artefactos que deben serializarse son:

- Modelo de clustering.
- Preprocesador tabular.
- Vectorizador textual.
- Modelo auxiliar KNN, si aplica.
- Reductor PCA o TruncatedSVD.

- Nombres de características.
- Columna textual usada.
- Tipo de dataset.
- Configuración del entrenamiento.
- Métricas obtenidas.

La serialización puede realizarse mediante herramientas como joblib o pickle.

Al clasificar un nuevo registro, el sistema debe cargar los artefactos previamente guardados y aplicarlos en el mismo orden utilizado durante el entrenamiento.

Este proceso es fundamental porque garantiza consistencia. Si se recalculara el escalador, el codificador o el vectorizador con datos nuevos, el espacio de características cambiaría y la predicción ya no sería comparable con los clusters originales.

FRONTEND

El frontend fue desarrollado utilizando el framework Angular, el cual permite construir una interfaz web moderna e interactiva. Este componente se encarga de:

- Proporcionar una interfaz gráfica para que el usuario pueda ingresar comandos.
- Permitir la carga masiva de comandos mediante archivos de texto.
- Mostrar la salida generada por el backend.
- Visualizar los resultados de las operaciones realizadas sobre el sistema de archivos.

La comunicación con el backend se realiza mediante solicitudes HTTP. Cuando el usuario ejecuta comandos desde la interfaz, estos son enviados al servidor para su procesamiento.

El frontend incluye los siguientes elementos principales:

- Área de salida del sistema
- Botones de ejecución de comandos
- Carga masiva de archivos de comandos
- Interfaz visual para mejorar la experiencia del usuario



TalentMosaic

Segmentación inteligente de freelancers y reseñas

Carga y preprocesamiento

Configuración y entrenamiento

Interpretación de los segmentos

Evaluación y validación

Clasificación de nuevo registro

Exportación de reportes

Carga y preprocesamiento

Sube los archivos CSV de freelancers y reseñas, realiza limpieza básica y visualiza una muestra de los datos.

Freelancers

freelancers

Carga y limpia el dataset de perfiles de freelancers.

Archivo CSV

Seleccionar archivo Sin archivos seleccionados

Nombre del archivo

freelancers.csv

Cargar CSV

Limpiar dataset

Ver preview

No hay datos para mostrar todavía.

Reseñas de clientes

reviews

Carga y limpia el dataset de reseñas textuales.

Archivo CSV

Seleccionar archivo Sin archivos seleccionados

Nombre del archivo

freelancers.csv

Cargar CSV

Limpiar dataset

Ver preview

No hay datos para mostrar todavía.



TalentMosaic

Segmentación inteligente de freelancers y reseñas

Carga y preprocesamiento

Configuración y entrenamiento

Interpretación de los segmentos

Evaluación y validación

Clasificación de nuevo registro

Exportación de reportes

Configuración y entrenamiento

Selecciona el dataset, algoritmo y parámetros para entrenar el modelo de clustering.

Parámetros de entrenamiento

Configura el dataset, algoritmo y parámetros del modelo de clustering.

Tipo de dataset

Freelancers

Nombre del archivo

freelancers_dev.csv

Algoritmo

K-Means

Random state

42

Número de clusters

3

Entrenar modelo

Modelo entrenado correctamente

Guarda el model_id para análisis, evaluación, clasificación y exportación.

MODEL ID

024be3ef-8748-451a-baa9-85db597ee5d6

freelancers

gmm

freelancers_dev.csv

Registros

1500

Features

1820

Clusters

3

Ruido

0

Silhouette Score

0.1698

Más alto es mejor.



TalentMosaic

Segmentación inteligente de freelancers y reseñas

Carga y preprocesamiento

Configuración y entrenamiento

Interpretación de los segmentos

Evaluación y validación

Clasificación de nuevo registro

Exportación de reportes

Interpretación de los segmentos

Visualiza los clusters en 2D, revisa perfiles, nombres de segmentos, términos relevantes y cruces entre modelos.

Seleccionar modelo

Ingresa un model_id entrenado o selecciona uno del historial local.

Model ID

024be3ef-8748-451a-baa9-85db597ee5d6

Modelos recientes

freelancers · gmm · freelancers_dev.csv

Cargar interpretación

Visualización PCA/SVD

No hay puntos para graficar.

Perfiles de segmentos

No hay perfiles para mostrar.



Evaluación y validación

Consulta métricas internas del clustering y compara modelos entrenados para justificar la configuración seleccionada.

Seleccionar modelo

El historial se guarda localmente con los modelos que has entrenado desde el frontend.

Modelo entrenado

freelancers · gmm · freelancers_dev.csv

Limpiar historial

Métricas del modelo seleccionado

Estas métricas permiten validar la calidad interna del agrupamiento.

MODELO SELECCIONADO

024be3ef-8748-451a-baa9-85db597ee5d6

freelancers gmm freelancers_dev.csv

Interpretación de métricas

Guía rápida para explicar los resultados durante la defensa.

Silhouette Score

Evalúa qué tan parecido es un registro a su propio cluster en comparación con otros clusters. Su valor suele estar entre -1 y 1. Mientras más cercano a 1, mejor separación existe.



Clasificación de nuevo registro

Asigna un freelancer o una reseña nueva al segmento existente que mejor le corresponde, sin reentrenar el modelo.

Nuevo registro

Ingresa los datos del freelancer o la reseña que deseas clasificar.

Tipo de registro

Freelancers

Modelo entrenado

gmm · freelancers_dev.csv

Model ID

024be3ef-8748-451a-baa9-85db597ee5d6

Datos del freelancer

Campos alineados con freelancers_dev.csv, excluyendo freelancer_id.

Proyectos completados

40

Ingresos totales

50000

Resultado de clasificación

Cuando clasifiques un registro, aquí aparecerá el segmento asignado.

Todavía no se ha clasificado ningún registro.



Descarga los resultados del clustering en CSV o un reporte PDF con gráficas, métricas y perfiles básicos.

Seleccionar modelo

Elige un modelo entrenado para exportar sus resultados.

Modelo entrenado

freelancers · gmm · freelancers_dev.csv

Model ID

024be3ef-8748-451a-baa9-85db597ee5d6

MODELO SELECCIONADO

024be3ef-8748-451a-baa9-85db597ee5d6

freelancers gmm freelancers_dev.csv

Registros

1500

Clusters

3

Ruido

0

Reportes disponibles

Descarga los resultados del modelo en CSV o PDF.



Exportar CSV

Descarga las columnas originales junto con el cluster asignado, nombre del segmento y coordenadas PCA/SVD.



Exportar PDF

Genera un reporte con información general, métricas, distribución de clusters y visualización PCA/SVD.

Descargar CSV

Descargar PDF